

Devoir 3 : Implémentation et comparaison des architectures RAG

Prof. Ikram BEN ABDEL OUAHAB

2026

L'objectif de ce TP est de :

- Implémenter plusieurs architectures RAG
- Tester et comparer leurs performances
- Analyser leurs limites et avantages
- Travailler dans un contexte applicatif réel

Instruction importante

Il faut appliquer toutes les architectures RAG étudiées dans ce TP sur **votre propre sujet de projet**. Travail individuel dans ce TP. Par la suite en votre groupes de projets, vous aurez par quoi commencer.

Consignes :

- Le dataset utilisé doit être adapté au domaine du projet
- Les requêtes doivent être cohérentes avec le sujet du projet
- Les tests doivent refléter un usage réel du domaine applicatif
- L'analyse finale doit être contextualisée au projet personnel

1 Préparation

1.1 Installation

Compléter les imports nécessaires :

```
# TODO : installer faiss, sentence-transformers, numpy, sklearn, transformers, et tout
```

1.2 Dataset

Remplacer le corpus suivant par un dataset lié au projet. Exporter sous format structuré CSV ou JSON.

```
documents = [  
    "Document 1 lié au domaine du projet",  
    "Document 2 lié au domaine du projet",  
    "Document 3 lié au domaine du projet"  
]
```

2 LLM sans RAG

Implémenter une fonction baseline :

```
def llm_no_rag(query):  
    # TODO : réponse sans retrieval  
    pass
```

Tester sur des requêtes liées au projet.

3 RAG Classique, naïf

3.1 Embeddings

```
from sentence_transformers import SentenceTransformer  
  
model = SentenceTransformer("all-MiniLM-L6-v2")  
doc_embeddings = model.encode(documents)
```

3.2 Retrieval

```
def retrieve(query, k=2):  
    # TODO : calcul similarité + top-k documents
```

3.3 Pipeline RAG

```
def rag_classic(query):  
    # TODO : retrieval + prompt + LLM
```

4 RAG avec Re-ranking

```
def rerank(query, docs):  
    # TODO : score plus précis des documents  
  
def rag_rerank(query):  
    # TODO : retrieval + reranking + réponse finale
```

5 RAG Hybride

```
def hybrid_retrieve(query):  
    # TODO : dense + sparse + fusion
```

6 Multi-hop RAG

```
def multi_hop_rag(query):  
    # TODO :  
    # étape 1 retrieval  
    # reformulation  
    # étape 2 retrieval  
    # réponse finale
```

7 Graph RAG

```
def graph_rag(query):  
    # TODO :  
    # exploration du graphe  
    # relations entre entités  
    # génération réponse
```

8 Agentic RAG

```
def agentic_rag(query):  
    # TODO :  
    # décision retrieval ou non  
    # boucle reasoning  
    # réponse finale
```

9 Résultats : Comparaison des architectures

9.1 Tests

```
queries = [  
    "Question 1 liée au projet",  
    "Question 2 liée au projet",  
    "Question 3 liée au projet"  
]
```

9.2 Tableau de résultats

```
results = {  
    "baseline": [],  
    "rag_classic": [],  
    "rag_rerank": [],  
    "rag_hybrid": [],  
    "multi_hop": [],  
    "graph_rag": [],  
    "agentic_rag": []
```

}

9.3 Métriques d'évaluation

Implémenter toutes les métriques d'évaluation vu en cours. Calculez les et interpréter les résultats.

9.4 Fonctionnalités clés :

- Visualisation des embeddings (t-SNE)
- Interface Gradio
- Intégration FAISS optimisé
- Utilisation API LLM (OpenAI / Mistral / Ollama) Interdite !

9.5 Questions

Répondre aux questions suivantes, selon votre sujet :

- Quelle architecture est la plus performante ?
- Quelle architecture est la plus robuste ?
- Quelle architecture est la plus adaptée au projet ?
- Quelle architecture produit le plus d'hallucinations ?

9.6 Livrables

- Notebook Jupyter (.ipynb)
- Code fonctionnel pour chaque architecture
- Tableau comparatif des résultats
- Analyse critique finale